Received 16 June 2025; revised 2 August 2025; accepted 14 August 2025. Date of publication 20 August 2025; date of current version 11 September 2025.

Digital Object Identifier 10.1109/OJCOMS.2025.3600706

Intent-Based Infrastructure and Service Orchestration Using Agentic-AI

DIMITRIOS BRODIMAS, ALEXIOS BIRBAS (Member, IEEE), DIMITRIOS KAPOLOS, AND SPYROS DENAZIS

Department of Electrical and Computer Engineering, University of Patras, 26504 Patras, Greece

CORRESPONDING AUTHOR: A. BIRBAS (e-mail: birbas@ece.upatras.gr)

The publication fees of this manuscript have been financed by the Research Council of the University of Patras.

ABSTRACT This paper introduces a novel framework that integrates agentic Artificial Intelligence (AI) with Intent-Based Networks (IBN) to enable autonomous management, configuration, and optimization of mobile network services and resources. Leveraging the advanced reasoning and natural language processing capabilities of an Large Language Model (LLM), the proposed architecture translates high-level user intents into precise network actions, facilitating user-friendly and scalable network orchestration. The framework employs a distributed multi-agent system, where specialized agents collaborate to decompose user intents, provide computational infrastructure, and deploy services using industry-standard Infrastructure-as-Code (IaC) tools. By supporting natural language interactions, the system reduces operational complexity and enhances accessibility for users with varying technical expertise. Experimental evaluations demonstrate significant improvements in task completion rates, response accuracy, and operational efficiency compared to traditional manual methods, particularly for complex network management tasks. In essence, this work creates an intelligent network orchestration framework that adapts to user needs by automatically configuring network and computing resources while operating with minimal human intervention.

INDEX TERMS Agentic artificial intelligence, intent based networks, infrastructure as code, model context protocol.

I. INTRODUCTION

THE RAPID evolution of mobile network technologies, particularly with the advent of 5G, has fundamentally transformed our digital landscape. This transformation has unlocked unprecedented possibilities across domains ranging from autonomous systems to smart infrastructures. However, with these advancements come significant challenges in network management, scalability, and customization. As networks become increasingly pervasive, there emerges a critical need to adapt network behavior to the specific and constantly changing requirements of diverse industries and applications.

Intent-based networking represents a promising paradigm shift, aiming to translate high-level user goals into network configurations, without needing to define the detailed steps required to achieve them [1]. Yet conventional orchestration approaches struggle to address the inherent complexities of today's dynamic network environments, especially as end-users increasingly lack specialized technical expertise. This gap necessitates a new generation of orchestration platforms that combine scalability and efficiency with the ability to interpret natural language inputs and transform them into precise network actions.

The recent breakthroughs in Generative AI (GenAI) and LLMs have presented a unique opportunity to intelligently transform natural language requirements into network configuration, paving the way for IBNs. However, the stochastic nature of LLMs makes them challenging to instruct while maintaining consistent, structured, and robust outcomes that can be directly fed to network controllers.

To address these limitations, the field has turned to Agentic AI, which can effectively contribute to evolving network management through intelligent, autonomous systems. Agentic AI refers to AIsystems with minimal human intervention, often in complex, real-world scenarios, through the use of AI agents. While the concept of AI

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

agents is not new, recent technological advancements have transformed them from theoretical constructs into powerful, practical tools that can bridge the reliability gap left by traditional LLMs. In their early stages, agents were limited to simple rule-based systems capable of performing only basic predefined tasks. The machine learning era marked a significant advancement, as agents gained the ability to learn patterns and enhance their performance over time. Today's AI agents represent the culmination of this evolution, as they are advanced, context-aware systems capable of complex reasoning and problem-solving across multiple domains.

At its core, an agent is a system that leverages an AI model to interact with its environment in order to achieve user-defined objectives. This interaction involves a sophisticated combination of reasoning, planning, and the execution of actions, often through external tools, to fulfill complex tasks [2]. The architecture of an agent can be conceptualized as comprising two interconnected components: the brain and the body. The brain, represented by the underlying AI model, serves as the reasoning center responsible for planning and determining appropriate actions based on contextual analysis. Meanwhile, the body encompasses the agent's operational capabilities and the tools it has been equipped with to interact with its environment. Crucially, the range of possible actions an agent can perform is directly dependent on the tools available to it, though it is important to distinguish that actions and tools are not synonymous, because a single action may involve the coordinated use of multiple tools to achieve a specific objective.

This sophisticated architecture enables modern AI agents to function as more than mere control mechanisms for physical infrastructure. Equipped with advanced reasoning and language understanding capabilities, these agents can effectively perceive network states, interpret user-defined intents, and perform complex tasks to fulfill those intentions. This agentic AI approach represents a fundamental paradigm shift in network management, where agents serve as intelligent decision-making entities that comprehend users' requirements and based on them can perform a wide range of control actions over the network, offering the reliability and consistency that traditional LLMs alone cannot provide.

Our research explores the integration of agentic AI into IBNs, presenting a novel architectural framework that leverages LLMs to enable autonomous management, configuration, and optimization of mobile network services and resources. We demonstrate how this approach facilitates truly dynamic, user-centric service delivery while simultaneously reducing operational complexity and enhancing adaptability across diverse network scenarios. By allowing users to interact with networks through natural language rather than technical specifications, we provide a more intuitive and accessible networking paradigm.

To that extent, this work proposes and evaluates a framework that capitalizes on the capabilities of the agentic AI, to deliver in an inclusive and user-friendly manner of the computational resources and services that serve as the backbone infrastructure of contemporary networks. Specifically, this framework supports the following:

- Intent decomposition and fulfillment through the collaboration of a team of highly specialized agents
- Seamless interaction with the infrastructure layer via industry-standard tools, implementing explicit IaC processes.
- Provisioning and orchestration of compute clusters, resources, and associated services in alignment with the interpreted intent.
- Comprehensive assessment of the system's performance using diverse and novel measurement criteria.

The rest of the paper is structured as follows: Section II describes the related work regarding the IBN and their evolution towards agentic AI. Section III presents the proposed framework in detail. In Section IV, the implementation details are presented. The evaluation methodology and the results, as well as an indicative proof of concept and the framework's limitations are included in Section V and a discussion about limitations of the framework. Finally, Section VI comprises the conclusion and future work.

II. RELATED WORK

Intent represents a set of clearly defined objectives and desired outcomes for a network, described using high-level language without specifying the implementation details. Intent can take various formats: it might be simple written instructions (such as "I want to add a second UPF to the 5G Core located in Patras"), visual representations like diagrams showing the desired outcome, or for experienced users, it can include additional technical files (like YAML or JSON documents) that provide the system with detailed and specialized network information. IBNs aim to simplify network management and operation by minimizing the need for manual user involvement. The core functionalities of IBN are generally categorized into two main areas: Intent Fulfillment and Intent Assurance. Intent Fulfillment involves the ingestion, interpretation, and orchestration of intent, focusing on delivering the necessary capabilities and interfaces to translate intent into effective configurations within the network (this paper specifically addresses this component). On the other hand, Intent Assurance encompasses the tools and mechanisms that verify whether the network successfully meets the objectives defined through Fulfillment processes [3].

IBNs are not a new concept with multiple implementations being demonstrated over the years [1], [4]. However, they attracted a growing research interest following the introduction of generative AI and LLMs. For example, the authors in [5] demonstrated how GenAI can effectively decompose user prompts into network configurations. Similarly, the NetGPT framework [6] presented a fine-tuned model capable of delivering personalized generative services tailored to network management requirements. From the industry's perspective, Ericsson's work on the evolution of service management highlighted the transition toward

intent-driven networks, emphasizing the shift from traditional configuration approaches to abstract intent specification [7]. Our previous works [8], [9], and [10] also align with those initiatives, mainly exploring the translation of the user intent into actual configuration for advanced operational support systems handling network resources across the edge-cloud continuum, by leveraging the capabilities of GenAI.

Apart from the growing interest of the industry and the academia, the usefulness of this proposition is also validated by the standardization bodies, which hasten to define how those solutions will interact with the actual infrastructure and legacy network controllers and drivers. According to European Telecommunications Standards Institute (ETSI) technical specification [11], the concept, architecture, and functional requirements are defined for intent-driven management, and are complemented by work in [12]. In this work where the enhancements of intent-driven management services identifying key issues and evaluating potential solutions are investigated. Complementing these, 3rd Generation Partnership Project (3GPP) has published extensive documentation [13] on intent-driven management, including references to deployment scenarios, planning approaches, and technical specifications. The TM Forum (TMF) has also been particularly active in this space, with their User Guide 921 [14] offering practical implementation guidance for Intent Management systems. This framework can be effectively integrated with other TM Forum standards such as TMF641 (Service Ordering) [15] and TMF645 (Service Qualification) [16] to create comprehensive intent-driven service management solutions.

Building upon these foundations, state of the art research and industry efforts have begun to explore Agentic AI, where autonomous, goal-driven agents extend the capabilities of generative models to enable more dynamic and context-aware network management. The authors of [17] introduced an agentic framework to configure SDN controllers interacting with simulated 5G networks, demonstrating that an agentic approach is feasible for the network management. The industry collaboration between Google and Ericsson is also aligned with the agentic evolution of IBNs. Their joint publication [18] showcases the design and implementation of a dynamic intent-based automation framework for 5G deployment, leveraging agents powered by Gemini [19]. This implementation showcased AI-driven closed-loop automation, where AI agents continuously monitor network performance against defined KPIs and proactively adjust resources to maintain service quality. This approach aligns with TM Forum's intent models and APIs, ensuring standardization and interoperability.

This paper embraces the agentic AI paradigm to deliver a practical implementation of an orchestration framework. Unlike prior works, our approach introduces several key innovations:

- Deployment of multiple autonomous, specialized agents to interpret user intents and manage the provisioning and configuration of required network. resources and services.
- Integration and benchmarking of various LLMs to power the agent behaviors.
- Fine-tuning of LLMs to enhance the framework's effectiveness and precision.
- Comprehensive evaluation of the system, including success rate, quality of generated configurations, user-facing responses, and overall value delivered to end users.
- Reliance on open-source and standards-based software for the implementation of the agentic framework, the agents' tools and the IaC processes.

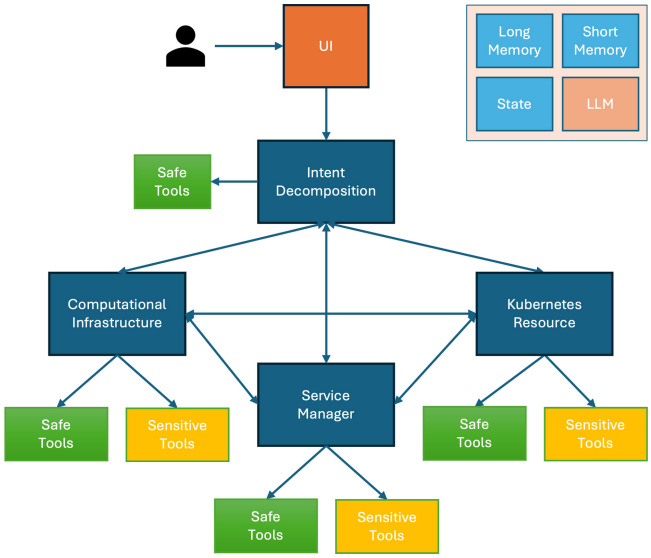


FIGURE 1. Agentic framework's architecture interconnected graph.

Overall, this work aims to showcase how AI-driven solutions can democratize access to network infrastructure and simplify the management of complex systems.

III. SYSTEM ARCHITECTURE

The framework (illustrated in Fig. 1) is implemented as a graph of sophisticated multi-agent system, designed to handle infrastructure management tasks through a distributed approach. The architecture centers around a user-driven intent decomposition mechanism that cascades into specialized management layers. The system demonstrates a thoughtful separation of tasks between infrastructure provisioning, cluster resources' management, and application deployment, with appropriate security categorization of tools at each level.

The graph's entry point is the User Interface (UI), which captures user prompts. These prompts are forwarded to the Intent Decomposition Agent (IDA), which analyzes and converts them into a concrete plan for specialized agents to implement. This agent then activates the appropriate specialized agent to begin addressing the problem. When a specialized agent cannot independently complete the

task, it may enlist other specialized agents for assistance. Alternatively, it can return to the IDA to provide additional task information or engage the user for clarification. Regarding the specialized agents' operational capabilities, they can utilize various tools from two distinct categories, the safe tools which operate autonomously, and the sensitive tools which require human verification before executing actions such as cluster deletion.

All agents share a set of core components that enable their functionality, including the framework's state, which serves as a message bus to manage communication between the agents. They also rely on the LLM, which functions as the agents' brain, generating thoughts, decisions, and action plans, as well as on both long-term and short-term memory systems that store interactions with users.

A. USER INTERFACE

The UI serves as the primary point of authentication and contact between users and the framework, designed to be accessible and functional for users of varying technical backgrounds. It implements a chat-like interface that enables natural, closer to human interaction.

Comprehensive session management capabilities give users full control over their interaction history. The system maintains conversation logs that users can reference, while also providing options to create new chat sessions or delete previous ones when no longer needed. This ensures a clean workspace while preserving important historical context when required.

The interface features advanced interaction capabilities, allowing users to rerun processing graphs from any point in the conversation history. This non-linear approach to conversation flow enables users to explore alternative paths or make corrections without starting over. Users can navigate effortlessly between different execution runs to compare outputs or review previous results. Additionally, the ability to edit any previously submitted message ensures flexibility and reduces friction when refining requests.

A key technical feature is the UI's support for human oversight mechanisms. The interface is designed to enable both "Human in the Loop" (active intervention during processing) and "Human on the Loop" (monitoring with selective intervention) approaches. This is accomplished through backend interrupt detection systems that identify when human input might be beneficial or necessary. The UI then presents appropriate insertion points where users can provide additional guidance, make corrections, or approve actions before processing continues.

B. INTENT DECOMPOSITION AGENT

Recent work in agentic AI has focused on architectural design, reflecting the field's shift toward developing autonomous systems capable of achieving complex goals [2]. These architectures range from simple single-agent setups, which work well for clearly defined tasks but struggle with flexibility, to more complex multi-agent systems that can scale better but require sophisticated coordination mechanisms [20], [21]. In the multi-agent category, current trends favor centralized approaches such as supervisor models or hierarchically organized agent structures, while fully distributed designs are beginning to emerge in recent implementations [22].

Following this research wave, IDA functions as the initial supervisor within a distributed agentic framework. Unlike traditional hierarchical systems, this agent adopts a distributed coordination approach. When a user submits a request to the UI, the IDA acts as the first point of contact with the graph, reasoning on the prompt, analyzing the overall intent and breaking it down into an action plan of high-level tasks that form the foundation for the rest of the agents' activity.

What distinguishes this supervisory agent is its limited scope of control. Rather than maintaining constant oversight throughout execution, it initiates the process by engaging the first appropriate agent in the planned workflow. From this point forward, the system operates on principles of agent autonomy and self-organization, with individual agents determining how to coordinate and collaborate until they complete their assigned objectives.

In this distributed model, the agents are designed with built-in collaboration capabilities. Each agent can independently reason over the provided task and determine when to call upon other specialized agents, share relevant context, or hand off completed sub-tasks. This peer-to-peer coordination eliminates bottlenecks that might occur with centralized oversight while promoting flexible, adaptive problem-solving approaches.

While the IDA does not actively monitor the entire execution process, it remains available as a resource if the agent network encounters significant obstacles or requires a high-level strategic guidance. In this case, the IDA can interrupt the operation of the framework in order to ask for additional information from the user.

The IDA also leverages a specialized vector store, through Retrieval-Augmented Generation (RAG), which contains examples of effective routing patterns and agent selection strategies, as well as information about the framework's overall capabilities and basic question-answering templates to facilitate a smooth discussion with the user. This knowledge base allows it to make informed decisions about which agent should be initially activated based on successful historical patterns for similar tasks. It also contributes to the security of the framework by containing examples that help the system avoid specific problematic behaviors, prevent responses to irrelevant prompts, and protect against itself the disclosure of sensitive data.

RAG combines LLMs generative capabilities with information retrieved from external knowledge sources in a three-step process: retrieval of relevant information from knowledge bases, augmentation of the prompt with this retrieved information, and generation of a response incorporating both the model's internal knowledge and the external

data. This approach addresses critical LLM limitations related to context window constraints by providing up-to-date information beyond knowledge cutoff dates, enabling access to specialized domain expertise without retraining, reducing hallucinations through factual grounding, also enhancing transparency via source attribution.

C. COMPUTATIONAL INFRASTRUCTURE AGENT

The Computational Infrastructure Agent (CIA) specializes in managing the complete lifecycle of computational resources within the framework. This agent handles all Create, Read, Update, Delete (CRUD) operations for infrastructure components requested by users, serving as the basis, bridging the user requirements with the actual infrastructure. Equipped with a comprehensive toolset for infrastructure management, the agent implements various IaC functions specifically designed for Kubernetes cluster orchestration. These tools enable automated provisioning, configuration, scaling, and decommissioning of the computing cluster infrastructure resources with minimal manual intervention. The Kubernetes framework was specifically chosen because it represents the state-of-the-art solution widely adopted by both the industry (e.g., Google, Microsoft) and the open-source community (e.g., the Linux Foundation) for hosting and managing computational and networking software and hardware resources, creating also a broader ecosystem known as the cloud-native ecosystem.

The agent also maintains persistent awareness of resource states through integration with a database system that functions as a comprehensive resource catalog. This catalog tracks all deployed clusters on a per-user basis, storing detailed specifications including worker node counts, resource allocation (Central Processing Unit (CPU), memory, storage), and the presence of any specialized acceleration hardware assigned to the clusters, such as a Graphics Processing Unit (GPU) or a Field-Programmable Gate Array (FPGA). When there is a user request, the CIA can reference this catalog to determine if existing resources can fulfill new requirements or if new resources need to be provisioned. For existing clusters, it can efficiently implement modifications to match changing user needs.

D. KUBERNETES RESOURCES AGENT

The Kubernetes Resources Agent (KRA) operates as a specialized component that can be activated only when a cluster already exists, in order to manage the internal Kubernetes resources. Once the CIA has successfully provisioned a Kubernetes cluster or when the system admin has manually added one, this agent takes over to handle the complex configuration and management of resources within that environment.

This agent is responsible for orchestrating the full spectrum of internal Kubernetes resources needed for application deployment and management. It handles the creation and configuration of pods, deployments, and stateful sets that host application components. The agent manages networking aspects by establishing appropriate services to expose applications internally or externally, configuring ingress controllers, and setting up network policies. Additionally, it implements security controls through the creation and configuration of ClusterRoles and ClusterRoleBindings, ensuring proper access management and permission structures within the cluster. What makes this agent particularly powerful is its foundation on a custom-trained model specifically fine-tuned on Kubernetes configuration datasets. This specialized training enables the agent to interpret user requirements and translate them accurately into precise Kubernetes resource definitions and kubectl commands. The model understands Kubernetes-specific concepts, best practices, and configuration patterns, allowing it to generate optimized resource definitions even for complex deployment scenarios.

Beyond just generating configuration specifications, the agent incorporates a code execution tool that can directly implement the generated kubectl commands on the selected clusters. This tight integration between understanding requirements, generating configurations, and executing commands creates a seamless workflow from user request to actual cluster configuration.

The KRA maintains awareness of cluster state throughout its operations, validating changes after implementation and providing feedback on successful deployments or any encountered issues. This ensures that users receive confirmation that their requested Kubernetes resources have been properly established and are functioning as expected within the broader cluster environment.

E. SERVICE MANAGEMENT AGENT

The Service Management Agent (SMA) functions as the dedicated deployment manager for user-requested services within the framework. Operating after cluster provisioning and basic Kubernetes configuration, this agent specializes in deploying and managing both application services and Kubernetes ecosystem extensions according to user specifications. Once a suitable cluster is available, the Service Agent can begin to implement the service layer that brings the actual functionality to the infrastructure.

The agent handles a diverse range of deployable components, including user applications (such as Web services, databases, or custom applications) or even specialized Kubernetes add-ons that extend cluster capabilities. These add-ons could include essential operational tools like Prometheus [23] monitoring stack for observability, Multus [24] for advanced container networking, storage solutions, or any other packages that enhance cluster functionality.

To execute these deployments efficiently, the SMA is equipped with three powerful tools designed to interact with external repositories:

1) A GitHub integration tool that enables direct access to user-authorized GitHub repositories. This allows the agent to retrieve application code, configuration files, and deployment manifests directly from version-controlled sources. Users simply need to provide appropriate GitHub access credentials, after which

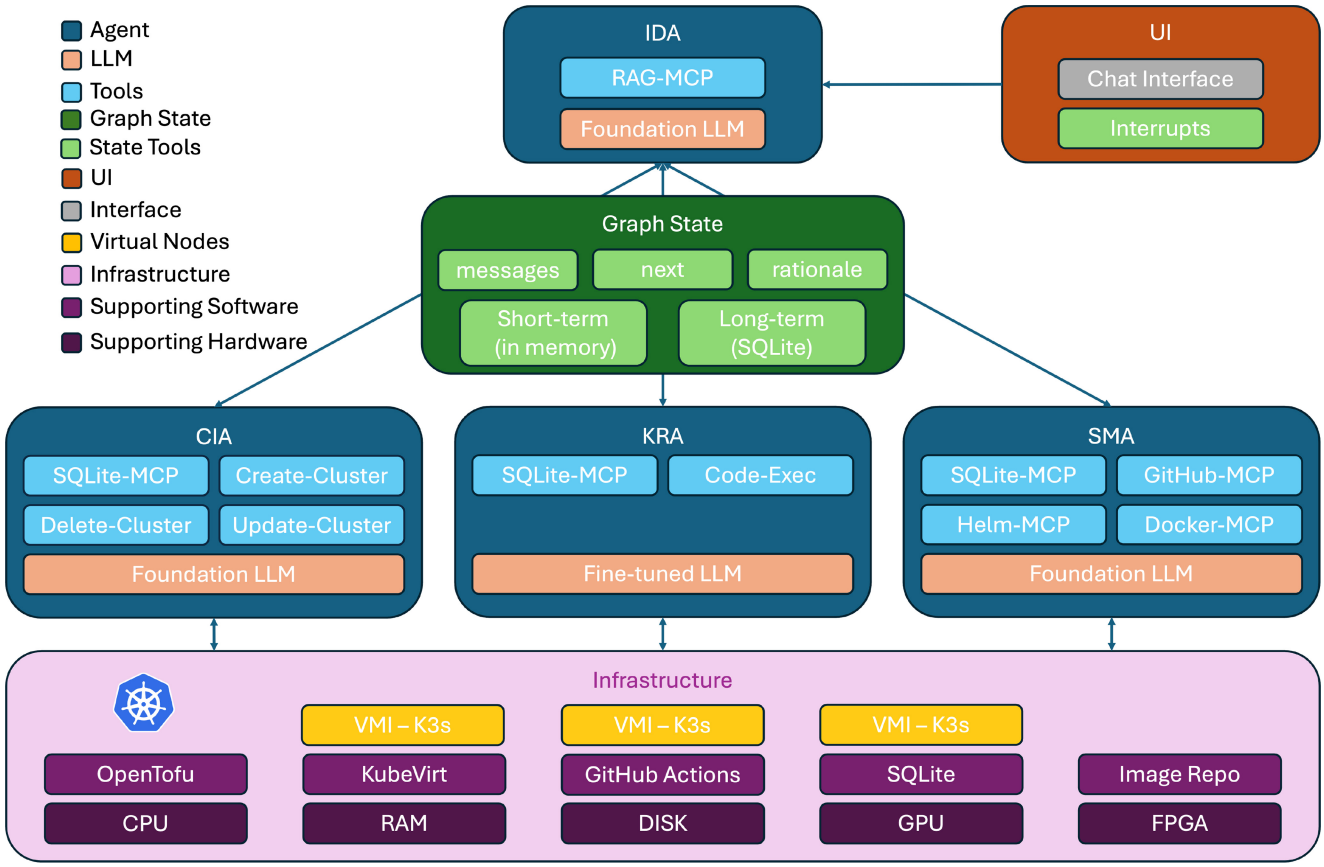


FIGURE 2. Technical representation of the Agentic AI framework.

the agent can deploy any applications stored in their repositories.

2) A Helm chart management tool that connects to various Helm repositories to deploy packaged Kubernetes applications. This capability streamlines the deployment of complex applications by utilizing pre-configured Helm charts, with users only needing to provide access to their preferred Helm repositories.
3) A Docker management tool which provides low-level access to running containers across the cluster, enabling the application's inspection and debugging on a container level.

IV. IMPLEMENTATION & EXPERIMENTATION

In this section, the different practical details of the framework will be presented on a bottom-up manner, as showcased in Fig. 2. Starting from the different tools (light blue boxes) and the different LLMs (orange boxes) powering the agentic framework, to the agents themselves (dark blue boxes), the graph's state and memory management (green boxes), and the infrastructure supporting this framework (purple and yellow boxes).

A. TOOLS

Building upon the previously established relationship between agents and their tools, the integration mechanism becomes a critical consideration. Tools serve as the primary means through which agents retrieve additional information and execute actions based on their analysis. However, connecting these diverse tools to agents requires a standardized approach to ensure seamless functionality.

The Model-Context Protocol (MCP) [25] has emerged as the predominant solution for this integration challenge. Developed as an industry standard by the agentic AI community, MCP provides a unified framework for building and deploying tools to the LLMs that power AI agents. This standardization is essential because it eliminates the complexity of custom integrations and ensures compatibility across different agent architectures. At its core, MCP implements a universal server-client architecture, as shown in Fig. 3, that establishes reliable connections between agents and their operational environment. This architecture encompasses both data sources, enabling agents to access the information they need for decision-making, as well as action capabilities, allowing agents to execute tasks within their designated domains. Through this standardized approach, MCP bridges the gap between an agent's cognitive capabilities and its practical functionality.

The first tool is a knowledge retrieval tool (RAG-MCP-Tool) used to enhance the IDA's decision-making by providing access to contextual information stored as emeddings within a ChromaDB [26] vector database. When

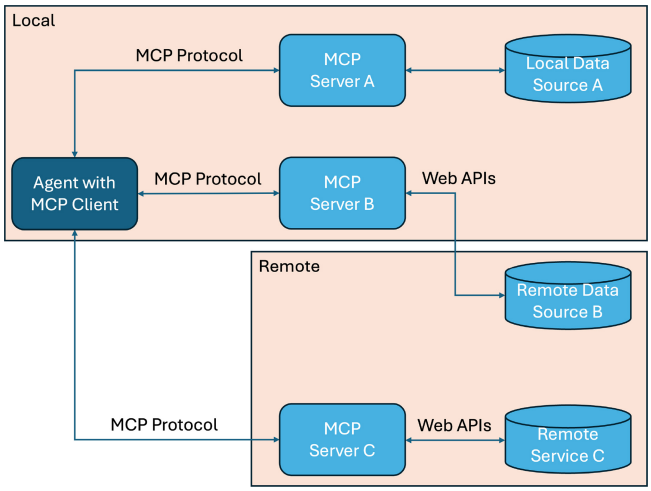


FIGURE 3. MCP architecture diagram.

the agent requires reference to relevant documentation or analogous past cases during the decomposition of a task, the tool utilizes the Facebook AI Similarity Search (FAISS) [27] algorithm to retrieve information. It is specifically configured to return the top two most relevant results per query, maintaining a balance between providing sufficient context while avoiding information overload. To populate ChromaDB, a diverse set of usage examples and routing scenarios, along with documents covering general networking and computing knowledge, are ingested through the Docling parser [28] in conjunction with EasyOCR [29]. The combination of the vector store and its population mechanism is critical for the agent, as it forms the foundation for few-shot learning influencing, in turn, the agent's decision-making and the reliability of its operations. The combination of the specific tools guarantee that the information is not corrupted during the loading phase and that it is reliably stored and retrieved.

The cluster registry tool (SQLite-MCP-Tool) serves as a centralized information repository that maintains comprehensive records of all Kubernetes clusters within the system. Powered by an MCP-integrated SQLite [30] database, this tool retrieves or updates detailed information about each cluster, including ownership (unique user number) and essential cluster details like the master node's Internet Protocol (IP) address and a securely encoded kubeconfig file enabling authenticated interaction with the cluster's API server. The resource specifications are also tracked by storing the flavor used to create the nodes, along with any specialized hardware like GPUs.

The framework is also boosted by three additional MCP tools to enhance the application management. The GitHub-MCP-Tool [31] establishes secure connectivity with users' GitHub repositories, enabling direct access to source code and deployment specifications, minimizing the time and effort for the user to deploy and handle its services. The Kubernetes-MCP-Tool [32] manages Kubernetes-specific artifacts and Helm Charts, providing a streamlined interface for deploying containerized applications within the cluster environment. The Docker-MCP-Tool [33] offers deeper container-level access, allowing users to interact directly with running pod containers for advanced debugging and troubleshooting purposes.

The framework features a specialized Cluster-Creation-Tool, Cluster-Deletion-Tool and Cluster-Update-Tool that provides users with on-demand cluster provisioning capabilities, based on the common IaC practices already adopted by the network industry. Initially, it establishes virtual machines on a host Kubernetes cluster using the cloud-native KubeVirt project [34]. This process is then automated through OpenTofu [35], which creates scripts that deploy customizable VMs (for example, Ubuntu 22.04 configurations with 2 vCPUs, 4GB vRAM, and 40GB storage) and installs the K3s cluster distribution, configuring these VMs as either master or worker nodes. This Day-1 installation and configuration of the VMs is the main reason for adopting OpenTofu, as it streamlines this process without requiring overly complex additional scripts. K3s on the other hand is selceted as it the most adopted and lightweight solution to provide cloud-native resources to the user. For cluster decommissioning, an OpenTofu script handles the removal process by deleting the specific namespace within the host Kubernetes cluster, which automatically terminates all VM instances that comprise the K3s cluster. In order to handle the worker subscription to the master and mainly the credential exchange between them (e.g., kubeconfig and cluster join tokens) the OpenTofu scripts run through GitHub Actions, enabling API-based access to these resources. The Cluster-Update-Tool works in a similar manner, creating or deleting worker nodes on specified clusters. On the fly configuration of the VMs to add or remove resources is not yet implemented without tearing down the VMs.

Last but not least, the Code-Exec-Tool serves as a critical execution interface for the framework, leveraging LangChain's Python REPL [36] capabilities to run Python scripts. This tool's primary function is to execute Python code that contain the kubectl commands generated by the finetuned model of the KRA. By doing so, it enables direct configuration and management of users' K3s clusters, forming an essential bridge between the intelligent agent's instructions and the actual implementation of those instructions within the cluster environment.

During the experimentation and paper writing phases, the Kubernetes-MCP-Tool provided a solid foundation for application deployment and management (e.g., pod, deployment, service creation or deletion), but fell short when it came to implementing complex cluster operations (e.g., scaling, role management). This limitation prompted the paper's authors to finetune a base model specifically addressing these advanced functionalities. Specifically, this tool could handle 38% of the related dataset [37] that was utilized to retrain the model. Similarly, the GitHub-MCP-Tool lacked GitHub Actions integration capabilities, leading to the development of a custom tool to properly interface with and leverage GitHub Actions functionality.

TABLE 1. Language models' characteristics.

Models	Parameters	Size	Context	API
o4-mini	N/A	N/A	128K	OpenAI
Claude-3.7-Sonnet	N/A	N/A	128K	Antropic
Qwen3:32b	32b	20GB	40K	Ollama
Devstral	24b	14B	128K	Ollama
Mistral	7b	4.1GB	32K	Ollama
Llama3.2	3b	2GB	128K	Ollama
Qwen3:0,6b	0.6b	523MB	40K	Ollama

B. LARGE LANGUAGE MODELS

During the experimentation phase, various models were employed to serve as the "brains" of the agents, with a primary focus on enabling tool calling, an essential capability for interacting within the framework's ecosystem. As a result, instruction-tuned models were prioritized, while models optimized solely for reasoning at the time of experimentation, such as DeepSeek-R1, were excluded despite their potential strengths. The selection included both open-source and commercial models from different providers, aiming to offer a comprehensive evaluation of their respective performance and functionalities.

To ensure diversity, models of varying sizes, large, medium, and small, were incorporated to assess how model's scale and parameter count influence the framework's performance, resource utilization, and associated costs. LangChain [38] was used to facilitate communication with APIs of proprietary models such as OpenAI's o4-mini [39] and Anthropic's Claude-3.7-Sonnet [40]. Similarly, LangChain enabled interaction with locally deployed models via Ollama, including Mistral's Mistral-7B [41] and Devstral [42], Meta's LLaMA3.2-3B [43], and Alibaba's Qwen3-32B [44] and Qwen2.5-coder-0.5B. For the local models the context window was reduced to 8K tokens to make sure that they fit into the memory of the GPU supporting the experimentation.

To enhance autonomous and effective Kubernetes cluster management capabilities, specialized and fine-tuned models were used for the specific task, rather than foundation models. This was done because the foundation models provided general guidelines and YAML files to implement the tasks which were subject to indentation errors, making them difficult to deploy. MistralAI's Mistral-7b-Instruct-v0.3 and Alibaba's Qwen3-0.6B models were fine-tuned using the 4-bit Quantized Low-Rank Adaptation (QLoRA) approach [45]. LoRA [46] is a Parameter-Efficient Fine-Tuning (PEFT) [47] technique designed to reduce computational complexity while achieving effective adaptation of LLMs to specific tasks. LoRA adds trainable low-rank matrices to the pre-trained weights of an existing LLM, freezing most of the original weights and significantly reducing the number of parameters that need adjustment during the training phase. In that way, the model can handle tasks from unfamiliar knowledge areas while maintaining all of its existing skills and abilities during the inference process.

The weight matrix in the pre-trained LLM is represented as $W \in \mathbb{R}^{d\times k}$, with low-rank update $\Delta W = BA$ where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, and $r \ll \min(d, k)$. During training, W_0 is frozen and does not receive gradient updates, while A and B are the ones getting trained. This leads to significant memory and compute savings while retaining performance. For $h = Wx$, the modified forward pass becomes:

$$h = Wx + \Delta Wx = Wx + BAx. \tag{1}$$

The quantization part, reduces memory usage by representing weights with lower precision. Specifically, it uses 4-bit NormalFloat (NF4) quantization, which is optimal for normally distributed weights. The quantization follows the following equation

$$Q(w) = \text{round}\left(\frac{w-\mu}{\sigma} \cdot \left(2^{b-1} - 1\right)\right) + 2^{b-1}. \tag{2}$$

where b=4 bits, μ and σ are the mean and standard deviation of the weight distribution.

The fine-tuning process leveraged a specialized dataset [37] consisting of 35 thousand examples of kubectl commands spanning all Kubernetes resources. Each dataset entry contained multiple features to provide comprehensive context: the command objective, an indicative name, the command syntax itself, a detailed description of required syntax and possible flags, and potential user queries that would necessitate the specific command. This rich feature set enabled the models to establish strong associations between user intent and appropriate kubectl commands. The dataset was partitioned following standard machine learning practices: 70% for the fine-tuning process and 30% reserved for post-training evaluation. The fine-tuning procedure utilized hybrid computational resources, combining both CPU and GPU processing.

C. AGENTS

All the agents of the graph are custom versions of the Reasoning and Acting (ReAct) agent [48] that can perform reasoning (following a chain of thought) and take action (creating plans or using assigned tools). The agent calls the language model with an input message list from the graph's state, and if the resulting AI Message contains any "tool calls", the agent will then execute the specific tool. When the tools are executed their responses are added to the list of messages as Tool Message objects. The agent then calls the language model again in the same way. The process repeats until no more "tool calls" are present in the response. This operation can be invoked in both synchronous and asynchronous manner, giving a more versatile usage of resources and handling of the graph flow. When any of the agents has finished its assigned tasks, it provides as an output a response to the user, the next agent needed to be invoked as indicated by the plan, and the rationale behind this decision.

```
You are a helpful assistant for managing
network and cloud computing services. Your
primary responsibility is to delegate user
queries to the appropriate specialized
assistants.

- For requests involving application
deployment or management, delegate to the
Service Assistant by invoking the
corresponding tool.
- For requests to create or delete Kubernetes
clusters, delegate to the Computational
Infrastructure Assistant.
- For Kubernetes resource management tasks
within a cluster, delegate to the Kubernetes
Resource Assistant.

You are not authorized to perform these
actions directly; only the specialized
assistants have permission to execute them.

The user is unaware of the internal
architecture or the presence of specialized
assistants. Therefore, do not mention them
explicitly. Instead, perform delegation
silently using function calls.

Your response must include only the name of
the assistant to which the task is delegated
to.
```

FIGURE 4. System prompt provided to the IDA.

```
You are a specialized assistant responsible
for managing Kubernetes clusters. The primary
assistant delegates tasks to you whenever the
user requests to create, delete, update, or
list Kubernetes clusters. Use the tools
assigned to you to complete these tasks.

Always respond in a short and concise manner.
```

FIGURE 5. System prompt provided to the CIA.

The IDA is guided by a specific system prompt, noted in Fig. 4. It is powered by one of the foundation LLMs and uses the RAG-MCP-Tool. This tool performs a retrieval step over provided examples before the agent makes any decisions or generates responses. In addition to this, the agent is aware of all the other agents in the system and can treat them as available tools. This design allows the agent to follow a ReAct-style reasoning and action approach. The output produced by this agent is always in a clearly defined and structured format. This format contains the agent's response, the name of the next agent to activate, as well as the rationale behind this decision, allowing the system to update the graph's state and route the flow accurately.

The CIA works in a similar manner as the IDA, also using a foundation LLM and following its own system prompt, found in Fig. 5. This agent uses the SQLite-MCP-Tool to check if the resources requested by the user already exist. It also uses the Create-Cluster-Tool to provision new clusters, including both master and worker nodes, based on user

```
You are a specialized assistant responsible
for managing Kubernetes resources within a
cluster.

When invoked, return only the exact `kubectl`
command required to complete the requested
task. Do not include any explanation,
formatting, or additional output.
```

FIGURE 6. System prompt provided to the KRA.

```
You are a specialized assistant responsible
for managing application deployments. The
primary assistant delegates tasks to you
whenever the user requests to create, delete,
update, or list applications. Use the tools
assigned to you to complete these tasks.

Always respond in a short and concise manner.
```

FIGURE 7. System prompt provided to the SMA.

```
class State(TypedDict):
   messages: Annotated[list[AnyMessage],
   add_messages]
   next: Optional[str] = None
   rationale: Optional[str] = None
```

FIGURE 8. Graph's state.

requirements. When a cluster is no longer needed, it can be removed using the Delete-Cluster-Tool. Whenever a cluster is created or deleted, the SQLite-MCP-Tool is used again to update the registry accordingly.

The KRA is unique in terms of using a fine-tuned model rather than a general-purpose foundation model. Its system prompt, indicated in Fig. 6, is designed to strictly instruct the model to return only the relevant kubectl command, with no extra explanation or surrounding text. Once the command is generated, it is passed directly to the Code-Exec-Tool, which executes it as-is. The kubeconfig file needed for the kubectl command to run on the correct cluster is obtained by the SQLite-MCP-Tool.

Finally, the SMA also operates using one of the foundation models and is guided by its respective system prompt, presented in Fig. 7. This agent is responsible for deploying, updating, and monitoring applications running on the provided Kubernetes clusters. To perform these tasks, it has access to several tools, including the Kubernetes-MCP-Tool, the Docker-MCP-Tool, and the GitHub-MCP-Tool. Similarly with the Kubernetes Resources Agent kubeconfig file is gathered by the SQLite-MCP-Tool.

D. GRAPH'S STATE

The overall operation of the graph is build around its state, which is composed by three attributes that control conversation flow and task execution. The state's actual definition is presented in Fig. 8. The first attribute, "messages", serves as the conversation repository, storing the complete interaction

history between user, the LLM, and any engaged tools. This collection maintains a chronological record of all internal graph events, communications and tool outputs. The second attribute, "next", functions as the control mechanism for the framework's execution flow. It contains a reference to the subsequent node in the graph that should be activated to progress the current task. The third one, contains the rationale behind the decision of which agent is selected as "next". The rationale is not presented to the user at any stage of interaction, but constitutes a very helpful tool that increases the trustworthiness of the system, as at any point of failure we know which decision led to it, enabling more effective debugging of the framework and context engineering. All these attributes concern a single invocation of the graph and are updated by agents using structured Pydantic [46] outputs, ensuring proper completion by the LLMs. Each output includes a brief description, usage examples, and allowed values. The LangChain framework handles this internally, embedding the details into the model prompt as instructions.

An indicative example of communication using the state that shows communication and coordination between agents, is the following where the system receives a user request to deploy a service and scale it up in a single prompt. The IDA processes the intent from the state's messages and reflects on it to generate a comprehensive execution plan. Upon completion, the IDA updates the messages object with its analysis and designates the SMA as the next agent in line (the rational is also updated whenever the agents take a decision, but it was omitted from this description to avoid repetition). The SMA subsequently retrieves the message history and evaluates the assigned task requirements. When sufficient information is available, the necessary tools are activated and the state is updated with the completion status, transferring control to the KRA for the scaling. However, if the SMA encounters insufficient information to proceed, it generates an inquiry message and redirects the workflow back to the IDA for clarification on the task or to the CIA to ask additional information for the deployment cluster. The KRA follows a similar pattern, analyzing the previous system state upon activation and engages its tools, or requesting specific information from the other agents. It is evident that the framework's ability to navigate the various workflow scenarios is fundamentally dependent on the guidance provided through the prompts and the constraints imposed by the structured output specifications.

For multiple interactions, the framework also features conversation memory capabilities, allowing it to maintain context by storing previous interactions within the graph's state. This state information is saved to an in-memory database through a threading mechanism, enabling the resumption of the conversation session as long as the thread remains active. The short-term memory is updated whenever the graph is invoked or provides the response to a message. The previous messages, if available in the session, are always loaded at the beginning of each new interaction with the graph.

To optimize resource usage, particularly regarding context window limitations, the system implements an automatic summarization process. When the conversation history reaches 50% of the available context window size (measured in tokens), an LLM generates a concise summary of the discussion. This summary replaces the original messages, which are then removed from short-term memory. This approach ensures the agent retains essential conversation history while avoiding context window constraints.

For long-term memory, an SQLite database is used to store the last summarized state of the short-term memory for each user for each conversation thread. This ensures the continuity of discussion and a more personalized interaction with the user.

Additionally, the graph's short-term memory functionality enhances user experience by supporting a "time-traveling" feature. This capability allows users to navigate back to specific checkpoints in the conversation, modify previously submitted prompts, and generate alternative conversation branches. By enabling users to revisit and alter earlier inputs, the system creates opportunities to explore different conversational directions and outcomes from the same starting point, effectively branching the dialogue into new pathways based on revised prompts.

The framework incorporates Human-in-the-Loop functionality through strategically placed interruption points within the graph's execution flow. These interrupts are specifically implemented before the system accesses sensitive nodes or critical tools within each agent's capability set. When such interrupts are triggered, the user interface displays a predefined notification that requests explicit user approval before proceeding with the potentially sensitive action. This creates a decision point where the user maintains control over critical operations. If the user grants permission for the action, the framework seamlessly resumes task execution from the precise checkpoint stored in memory, continuing along its intended path. However, if approval is denied, the system immediately terminates the current task and generates a notification informing the user that the operation has been aborted.

E. USER INTERFACE

For the purposes of this paper, intent is limited to written text entered through the UI as prompt. Additional text files can be included by copying and pasting their contents into the chat alongside the main request. Thus, the framework's UI is a simple implementation designed to support experimentation through a chat interface. Its most advanced features are buttons that appear when hovering over a specific message, allowing users to resubmit or edit it and restart the conversation from that point. Additionally, the previously mentioned interrupt mechanism is implemented as two buttons for accepting or rejecting a specific tool invocation. There are also plans to add features that allow users to modify the input to the tool prior to confirming its execution, as well as uploading mechanisms to extend the different file formats

acceptable from the framework. In the case of images, also multi-modal LLMs will be considered to either transform the image to text and then concatenate it to the prompt, or directly processing it along with the text.

In terms of security, the UI implements a basic authentication system using username and password combinations to keep user chat sessions private. When a user creates their credentials, the system generates a unique key identifier that serves two purposes, granting access to the previous chat sessions stored in long-term memory, and it provides access to the computing and network resources allocated by that user.

F. SUPPORTING INFRASTRUCTURE

The experimental testbed comprises a four-server cluster with 48 CPUs, 56GB RAM, 3.5TB storage, two NVIDIA graphics cards (RTX A5000 and A4000), and an AMD Zynq UltraScale+ FPGA. This infrastructure runs on standard Kubernetes and serves as the host environment for the K3s clusters provided by the framework. All system components—including databases, the framework itself, the user interface, Ollama, and AI models—are deployed as Kubernetes resources across this cluster. The RTX A5000 GPU is used to accelerate the operation of the medium sized models (Qwen3:32b & Devstral).

V. POC, EVALUATION, RESULTS & LIMITATIONS

A. PROOF OF CONCEPT

To validate the operational capabilities of our agentic AI framework, we conducted a practical demonstration showcasing its ability to autonomously provision cloud infrastructure and deploy complex network applications with minimal user interaction. This demonstration follows a similar methodology to our previous work [9], targeting the deployment of the Open5GS [49] 5G Core alongside the UERANSIM [50] RAN simulator.

The user intent involves deploying three Kubernetes clusters: one hosting the 5G Core, which supports two network slices (configured with Access Point Name (APN) “Internet” and “ims”), and two clusters each hosting a Next Generation Node B (gNB) instance connected to a respective slice. Two simulated User Equipment (UE) devices connect through the gNBs to the core network to access Internet services. To enable this setup, the Access and Mobility Management Function (AMF) and User Plane Function (UPF) ports must be exposed as NodePorts via Kubernetes services to support N2 and N3 interface traffic.

Currently, both Open5GS and UERANSIM lack external APIs for runtime management (with the exception of the subscriber database in Open5GS). Moreover, any configuration change to their network functions requires a complete restart of the respective component. Given these constraints, the development of a dedicated management tool (such as MCP or a custom solution) is considered out of scope for this document. Instead, we leveraged highly customizable Helm charts for both components, enabling the configuration of advanced scenarios by dynamically passing values such as the number of UPFs, supported slices, Public Land Mobile Network (PLMN), and more.

The user’s intent can be submitted either as a single prompt through the user interface or interactively through a multi-step conversation with the system. Thanks to the long-term memory functionality, the framework can maintain context across multiple interactions. In the following example, we illustrate the conversational mode to showcase this memory capability.

The user begins by logging into the system and requesting the provisioning of three Kubernetes clusters, each with one worker node. The IDA receives this input and consults the RAG-MCP-Tool to retrieve similar past requests. Based on these examples, the IDA formulates a plan, which includes invoking the CIA with a message to create the clusters. Upon receiving the task, the CIA reasons over the request and decides to execute the Cluster-Creation-Tool three times, triggering three GitHub Actions that provision a total of six VMs (each with K3s pre-installed). After each cluster is created, the SQLite-MCP-Tool updates the system’s database with the new cluster information. The user receives a success message detailing the cluster setup, and this information is also stored in the long-term memory database alongside the original prompt.

Next, the user prompts the framework to deploy the 5G Core on one of the clusters, apply the aforementioned slicing configurations, and expose the necessary ports. A URL pointing to the Helm chart repository of the 5G Core is also provided. The IDA processes this new prompt, enriched with the prior conversational history, and queries the vector store for relevant patterns. It then generates a new plan: engage the SMA to deploy the network functions and the KRA to expose the required ports.

SMA, upon activation, uses the SQLite-MCP-Tool to retrieve the target cluster’s details. It then leverages the Kubernetes-MCP-Tool to deploy the relevant Helm charts (including the control plane, UPF-1, and UPF-2), passing in specific configuration parameters such as APN support, Session Management Function (SMF) routing for those APNs, and the internal IP addresses of the UPF instances. After the SMA completes its deployment tasks, it confirms the plan and invokes the KRA.

KRA receives the task to expose the AMF and UPF ports, gathers the cluster information from the SQLite-MCP-Tool, generates the necessary Kubernetes commands, and executes them using its Code-Exec-Tool. Upon successful execution, a confirmation message is sent to the user and logged in the memory database. This process is illustrated in Fig. 9, to present how the framework operates in a complex prompt engaging multiple agents.

The final phase involves deploying UERANSIM. This follows the same process as the core deployment. SMA is triggered again to deploy the RAN components using Helm charts, configuring each gNB with the IP addresses of the corresponding AMF and UPF. Finally, the UE instances are

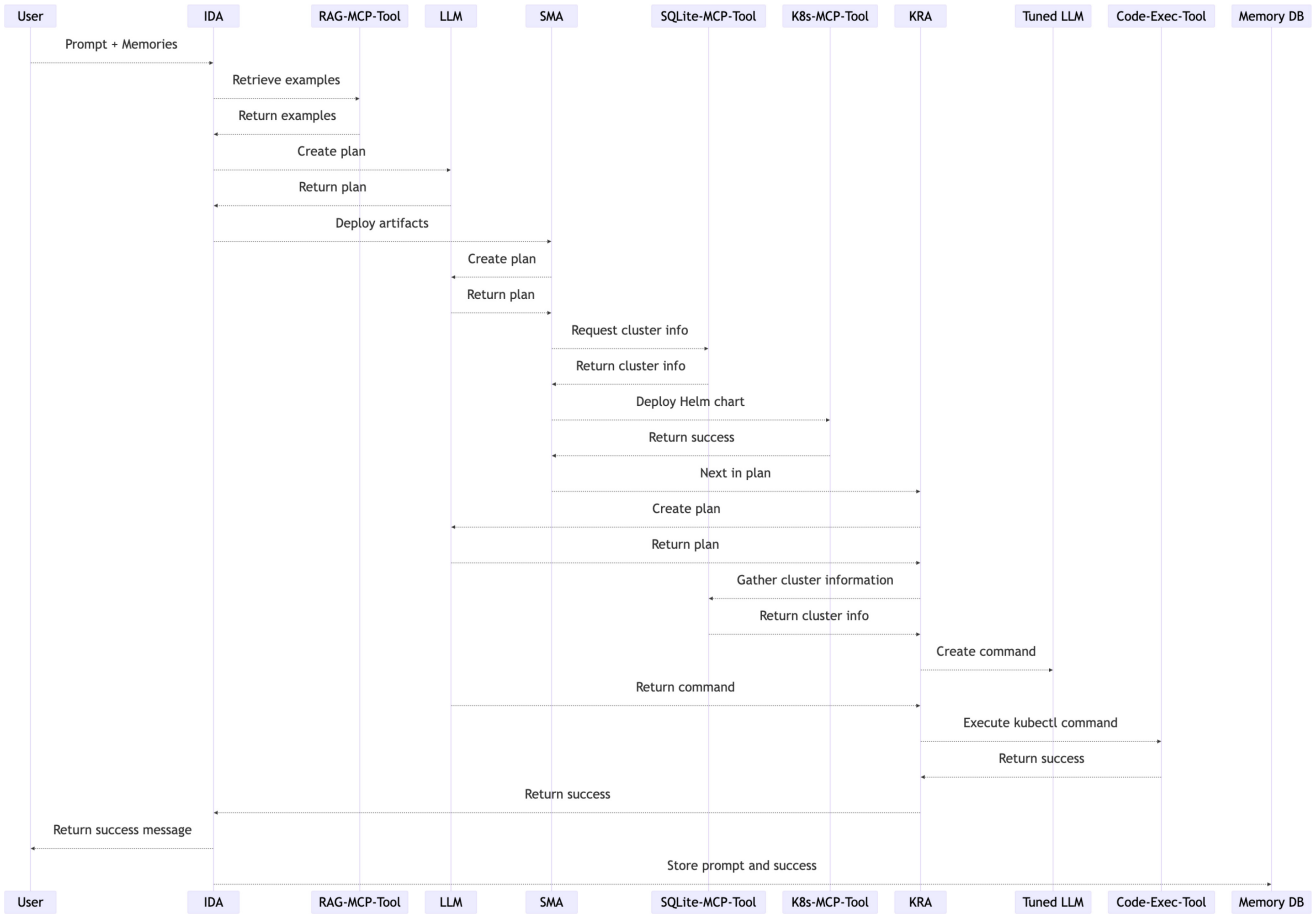


FIGURE 9. Sequence diagram of interaction with the framework.

deployed within the same clusters as the gNBs and connected accordingly. The PoC concludes with successful connectivity tests confirming that the UEs can reach external Internet endpoints via the 5G Core.

Selected screenshots demonstrating the framework's UI and outcomes achieved throughout this PoC are presented in the Appendix section.

B. EVALUATION METHODOLOGY

This subsection presents a comprehensive assessment methodology of the proposed multi-agent architecture framework. The framework's effectiveness is measured using various metrics that focus on five key operational areas:

1) The total number of *tasks completed successfully* and the associated *operational expenses*.
2) The accurately *decomposed user intentions* by the IDA.
3) The *latency* of framework from the provision of the intent up to the answer to the user, as well as the *operational costs* of use.
4) The response quality of the KRA, measured by *word-level similarities*.
5) The response quality of both the CIA and the SMA, assessed through *semantic similarity analysis*.
6) The performance improvement in *success rates* and *required time* for each task when the user utilizes the framework versus operating without it.

The baseline decisions and performance of the LLMs inside the framework are assessed by checking how well they operate in decomposing tasks (routing the traffic between the available agents) and in selecting specific tools to complete the tasks. The overall task completion rate is also measured, as well as the response latency of the time elapsed between query submission and the delivery of the completed task. Capital Expenditures (CapEx) is also considered as an estimate of the upfront cost needed to acquire the computing and storage resources required to host the framework, and particularly the LLMs powering it. Additionally, the Operational Expenditures (OpEx) of usage are gathered which include the financial impact of API calls to the models, taking into account how often the APIs are called and the number of tokens consumed across all agents, across all the testing data. This measurement gathering is performed using advanced observation and tracing tools like LangSmith [51] and LangFuse [52]. In addition to that, these two tools also contribute to the trustworthiness of the system, keeping detailed traces and logs of all agentic operations that the network engineers can use to keep track of any

data drifts, identification of points of failure and continuous context engineering.

The next phase involves validating the responses generated by the framework's LLMs for both user interactions and infrastructure-level outputs. This validation uses two approaches: word-level similarity and semantic similarity. Word similarity compares the generated output to the expected response on a textual level, which is critical for deterministic tasks requiring exact syntax, particularly when outputs serve as inputs to controllers and infrastructure components. Semantic similarity, on the other hand, evaluates whether the generated response conveys the intended meaning rather than matching exact wording. This approach is more suitable for assessing unstructured outputs mainly targeting user interactions. Essentially, meaning similarity is a way to relax the constraints of the evaluation, in order to tackle the stochasticity in the generation process of the LLMs. For structured processes requiring precise formatting, semantic similarity is not applied, as these scenarios demand the stricter constraints that the aforementioned exact textual matching provides.

Bilingual Evaluation Understudy (BLEU) [53], Recall-Oriented Understudy for Gisting Evaluation (ROUGE) [54], and Metric for Evaluation of Translation with Explicit ORdering (METEOR) [55] are used as word similarity metrics specifically to evaluate the foundation and fine-tuned models of the KRA, which generates precise kubectl commands for direct deployment. BLEU is a precision-based metric commonly used in machine translation and text generation evaluation. It compares n-grams in the generated (candidate) text against one or more reference texts. Modified n-gram precision (refer to (5)) is computed, where counts are clipped by the maximum number of occurrences in any reference. A brevity penalty (refer to (4)) is applied to penalize overly short translations.

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{3}$$

where w_n is the weight for each n-gram level (typically $w_n = \frac{1}{N}$). The brevity penalty is equal to:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-\frac{r}{c}} & \text{if } c \leq r \end{cases} \tag{4}$$

where c is the length of the candidate translation, and r is the length of the closest reference. p_n is the modified n-gram precision defined as:

$$p_n = \frac{\sum_{\text{ngram}\in\text{Cand}} \min(\text{C}_{\text{cand}}(\text{ngram}), \max_{\text{ref}} \text{C}_{\text{ref}}(\text{ngram}))}{\sum_{\text{ngram}\in\text{Cand}} \text{C}_{\text{cand}}(\text{ngram})} \tag{5}$$

where $C_s(y)$ is the number of appearances of s as a substring of y. BLEU scores range from 0 to 1, where 1 indicates a perfect match.

Similarly, ROUGE is a set of metrics that measure the quality of generated text by comparing it to reference texts using recall-oriented overlap statistics. The most common variants include ROUGE-N, which evaluates n-gram recall. In the specific experimentation of this paper ROUGE-1 is employed for the evaluation narrowing down equation (6) only to unigrams.

$$\text{ROUGE-N} = \frac{\sum_{\text{ngram}\in\text{Ref}} \min(\text{C}_{\text{cand}}(\text{ngram}), \text{C}_{\text{ref}}(\text{ngram}))}{\sum_{\text{ngram}\in\text{Ref}} \text{C}_{\text{ref}}(\text{ngram})} \tag{6}$$

ROUGE scores range from 0 to 1, with higher values indicating greater recall and similarity to the reference text.

METEOR aligns the candidate and reference text using exact, stem, synonym, and paraphrase matches. It calculates a harmonic mean of unigram precision and recall, with higher weight given to recall, and includes a fragmentation penalty for non-contiguous matches.

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \textit{Penalty}) \tag{7}$$

with

$$F_{\text{mean}} = \frac{10 \cdot P \cdot R}{R + 9P} \quad \text{and} \quad \textit{Penalty} = 0.5\left(\frac{ch}{m}\right)^3 \tag{8}$$

where P is unigram precision, R is unigram recall, m is the number of matched unigrams, and ch is the number of chunks. METEOR scores range from 0 to 1, with 1 representing perfect alignment.

Semantic similarity approaches like BERTScore [56] and an LLM-as-a-Judge [57] are applied to the CIA and SMA to ensure that their responses to users are contextually accurate. BERTScore uses contextual embeddings from a pretrained Bidirectional Encoder Representations from Transformers (BERT) model to measure similarity between candidate and reference texts. Instead of relying on exact token overlap, it evaluates semantic similarity by aligning words using cosine similarity in embedding space.

$$\text{BERTScore} = \frac{2 \cdot P \cdot R}{P + R} \tag{9}$$

where P (precision) and R (recall) are based on cosine similarities of matched tokens in the embedding space. BERTScore is typically reported as an F1 score and ranges from 0 to 1.

For the LLM-as-a-Judge, OpenAI's GPT-4-turbo was employed equipped with the prompt found in Fig. 10 which implements equation (10). Each response is evaluated on three criteria $C \in \{\textit{Accuracy}, \textit{Relevance}, \textit{Completeness}\}$. Each criterion $c \in C$ is scored as $S_c \in \{1 = \text{Poor}, 2 = \text{Fair}, 3 = \text{Good}\}$, and the final score can be measured using equation (10) and its value ranges from 1 to 3.

$$S_{\text{total}} = \frac{1}{|C|} \sum_{c \in C} S_c \tag{10}$$

In the results subsection, both word and meaning similarities are presented using percentages for better readability.

Beyond technical performance, the framework is also evaluated in terms of the value it adds for users, whether

```
You are an impartial, expert judge. Your role
is to critically and fairly evaluate the input
 provided, based on the specific criteria
outlined below. Use structured reasoning and
be objective and rigorous in your evaluation.

Provide a score based on the following scale:
'1 = Poor', '2 = Fair', '3 = Good'

Use the following criteria:
'Accuracy: Is the content factually correct?'
'Relevance: Does the response directly address
 the prompt or user request?'
'Completeness: Does it cover all necessary
aspects without missing key points?'
```

FIGURE 10. System prompt provided to the LLM-as-a-Judge evaluator

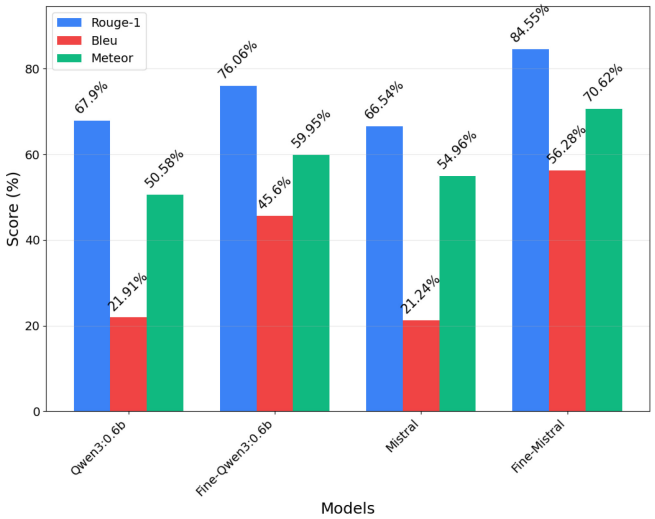


FIGURE 11. Word similarity evaluation.

they're administrators managing infrastructure or developers deploying applications. This part of the evaluation relies on qualitative criteria and a more abstract methodology. We compare traditional/manual processes for resource and service management with those using OpenSlice [58] orchestrator and finally the proposed agentic framework.

OpenSlice is an orchestrator adopted by the ETSI community [59], implementing an open-source, service-based Operation Support System (OSS) platform to enable Network-as-a-Service delivery. For this work, we utilized the OpenSlice instance deployed in the Patras5G testbed [60], [61], which supports two custom service orders: Kubernetes-as-a-Service and 5G-as-a-Service. These services are specifically tailored to the capabilities of the testbed. The OpenSlice user interface (UI) served as a reference point for evaluating and comparing the intent-driven interaction model of our agentic framework.

Five individuals from different professional levels, certified Kubernetes administrators, junior engineers, and senior stakeholders, all familiar with the relevant terminology were asked to complete a series of tasks thrice, once manually, once using OpenSlice, and once using the agentic framework. The criteria and metrics that were used for this specific evaluation are the following: (i) the time needed for the users to complete the task manually and create the needed prompts to pass them to the framework, and (ii) the success rate of correctly solving the problems. The provided tasks were grouped into three levels of difficulty: easy (such as scaling pods), medium (such as deploying and configuring clusters), and hard (such as setting up network policies or Role-Based Access Control). These tasks closely resemble, or are identical to those used as the "golden dataset" in the earlier stages of evaluation. For this experiment, KRA was powered by the fine-tuned Mistral and the rest of the agentic framework by Devstral.

C. RESULTS

Fig. 11 offers a detailed analysis of the performance improvements across all evaluated metrics for the fine-tuned models. Notably, the fine-tuned Mistral model achieves particularly strong results, and by taking into consideration variations in flag placement within commands (e.g., 'kubectl get pods -n test' is considered different from 'kubectl get pods –namespace test' and 'kubectl get -n test pods') which are treated as distinct expressions, the numbers increase to figures close to 90%, 65%, and 80%, respectively for BLEU, ROUGE-1 and METEOR metrics. All subsequent results are based on the use of the fine-tuned Mistral model to power the KRA.

TABLE 2. Successful intent decomposition, task completion, latency, operational and capital expenditures per model.

Model	Decomposition	Task	Latency	OpEx	CapEx
o4-mini	96.55%	93.10%	13.56s	1.11$	150$
Claude-3.7	93.12%	86.20%	11.32s	7.11$	150$
Qwen3:32b	93.01%	94.25%	34.55s	0	1200$
Devstral	94.83%	82.76%	5.07s	0	1200$
Mistral	57.47%	44.82%	2.22s	0	900$
Llama3.2	62.07%	37.93%	1.47s	0	900$
Qwen3:0.6b	42.53%	63.22%	2.01s	0	700$

Following, Table 2 presents the core evaluation of the framework regarding the task completion and efficient routing among the agents. The two proprietary models along with Qwen3:32b and Devstral clearly outperform all other models in both successful decomposition and task completion rates. However, this superior performance comes at a significant operational cost for the commercial models. For the smaller models, performance tends to decline as the parameter count decreases. It is important to highlight a key limitation of the local models: their context window is capped at 8K tokens. When this limit is exceeded, the GPU's available memory becomes insufficient, leading to increased latency and, in some cases, code execution failures. As a result, these models' effectiveness was constrained by their limited token capacity, which restricted their ability to reason, incorporate system prompts and example of usage retrieved using the RAG, process tool definitions and arguments, and generate

TABLE 3. Meaning similarity per model.

Model	BERTScore	LLM-as-a-Judge
o4-mini	75.30%	2.32
Claude-3.7-Sonnet	75.68%	2.48
Qwen3:32b	75.88%	2.37
Devstral	75.80%	2.41
Mistral	76.53%	2.17
Llama3.2	75.55%	1.83
Qwen3:0.6b	76.59%	2.4

TABLE 4. Comparison of manual and agentic usage in terms of time and success rate.

	Manual		OSL		Agentic	
Difficulty	Time	Success	Time	Success	Time	Success
Easy	30s	83%	8s	35%	10s	91%
Medium	4min	60%	12s	46%	35s	79%
Hard	9min	48%	20s	20%	1min	68%

coherent output. This makes the performance of Qwen3:32b and Devstral even more astonishing. In terms of latency, the proprietary models slightly under perform in comparison with the medium and small models, with the exception of Qwen3:32B, which shows notably higher latency. However, when the context window of small and medium models is expanded, their latency increases substantially, making the comparison more equitable.

Regarding response quality, all models provided highly satisfactory answers to the user, as illustrated in Table 3. Llama3.2 is the only model that performs notably worse than the others in this category. It is important to note that this evaluation was conducted solely on tasks that were successfully completed, implying that larger models had more data available for comparison, an observation stemming from the earlier findings.

Proceeding to the comparison between manual and Openslice service and resource management versus using the proposed framework, as presented in Table 4, the agentic framework proves increasingly beneficial as task complexity grows and deeper expertise is required. For network experts, simple and moderately difficult tasks are generally straightforward and can be completed quickly. However, less experienced users often needed additional time to consult documentation or clarify terminology. As task difficulty increased, the manual approach demanded significantly more time and multiple attempts to achieve the desired outcomes, resulting in reduced overall efficiency.

In terms of success rate, the agentic framework outperformed manual methods, though its effectiveness was influenced by the user's ability to understand the system and craft appropriate prompts. Nevertheless, the framework's built-in knowledge of the underlying infrastructure enabled more accurate interpretation and execution of the tasks. It is also worth noting that during the manual experiments users frequently turned to online LLMs for assistance in completing their tasks.

With respect to OpenSlice, the use of its UI and drop-down fields significantly accelerates the provisioning process. Its functionality is constrained though to the two predefined service orders. This constraint enables even non-expert users to complete those specific tasks with high accuracy (achieving a 100% success rate), but when users were instructed to perform more generic tasks beyond the predefined options and available fields, the system's flexibility was limited, resulting in poor outcomes. Furthermore, the execution times reported in Table 4 reflect scenarios with administrative privileges. Under standard user permissions, all deployments and service order modifications require administrator approval, which introduces substantial delays in task completion.

D. LIMITATIONS

While the proposed agentic AI framework demonstrates significant potential for intent-based network orchestration, several critical risks and limitations must be acknowledged and addressed for deployment in a production system.

The system's dependence on LLMs creates a risk of generating false or inaccurate information, especially when user instructions are unclear or vague. Even with careful prompt design and model retraining, these models might produce incorrect decisions and commands, resulting in unwanted changes to infrastructure, like incorrectly set up clusters or misconfigured service deployments. Human-in-the-loop, enabled by the interrupting mechanism of the framework, along with the structures outputs of the LLMs were introduced to mitigate this issue for critical operations, such as deleting clusters, services, and resources. Nevertheless, more sophisticated guardrails need to be introduced to ensure that no operation will violate the user's needs, even for the safe operations. Such guardrails include input validation and sanitization prior to agent engagement, secure APIs for inter-agent and tool communication, as well as enhanced tracing and monitoring of agent outputs to perform based on them continuous context engineering and thus increase the trustworthiness of the system [62], [63].

Furthermore, the limited context windows in local models at 8K tokens, constrain the reasoning capability of the agents under complex multi-step tasks. This limitation may cause reduced performance as network systems grow in complexity, unless carefully managed. While using more powerful hardware or proprietary LLMs could mitigate this issue, these solutions would increase either initial or operational expenses, respectively. Hence, this issue does not have a predefined solution, but instead requires a solution tailored to the particular use-case and infrastructure to be used.

The proposed framework exhibits an additional constraint related to inter-agent communication reliability and robustness. The current implementation employs a simplified state object architecture that demonstrates satisfactory performance within controlled deployment environments, particularly when all agents operate within a single server or a containerized pod. However, this approach presents

limitations when considering horizontal scaling scenarios involving increased agent populations or distributed deployment architectures across multiple host systems. To address these architectural limitations, industry standards such as the Agent-to-Agent (A2A) communication protocol [64], developed by Google and adopted by the Linux Foundation, or alternative frameworks like Agntcy [65] or Agent Communication Protocol (ACP) [66], which were not publicly available during the implementation phase, could provide robust foundational infrastructure for enhanced framework operation.

VI. CONCLUSION

This work introduced a comprehensive framework that applies agentic AI principles to intent-based orchestration of infrastructure and network services. By leveraging a distributed, multi-agent architecture and powerful LLMs, the system allows users to provision, configure, and manage Kubernetes clusters and services through natural language interactions. The framework not only translates user intent into structured and well defined configurations, but does so by engaging different highly specialized agents for the different actions needed to fulfill the intent.

The design emphasizes system prompt engineering, controlled execution flow through graph state management, and efficient handling of LLM context windows to maintain coherent reasoning and action. These foundational engineering decisions enabled robust tool invocation, model-guided orchestration, and reliable output delivery, even in complex use cases involving infrastructure provisioning, Kubernetes resource management, and application deployment.

Experimental evaluation across multiple LLMs, including fine-tuned models, confirmed significant improvements in word and semantic similarity metrics. The proposed agentic approach demonstrated high success rates in task execution, notably outperforming traditional manual methods in both efficiency and usability, particularly for complex tasks. The system was validated not only technically but also through comparative user studies that reinforced its accessibility and value to both experts and non-experts on the domain.

Finally, many of the framework's current limitations, as discussed in the previous section, can be addressed by leveraging its modular architecture to introduce dedicated network-domain agents. These agents would incorporate specialized tools designed to interface with exposed network elements such as the Network Exposure Function (Network Exposure Function (NEF)) in the 5G Core domain, the APIs of TerraFlowSDN [67] in the transport domain, and the interfaces provided by the Service Management Orchestrator (SMO) layer in the RAN domain. These architectural extensions would significantly enhance the framework's capabilities, positioning it to compete with advanced and mature network orchestrators such as ONAP [68], and ultimately enabling a truly end-to-end orchestration solution.

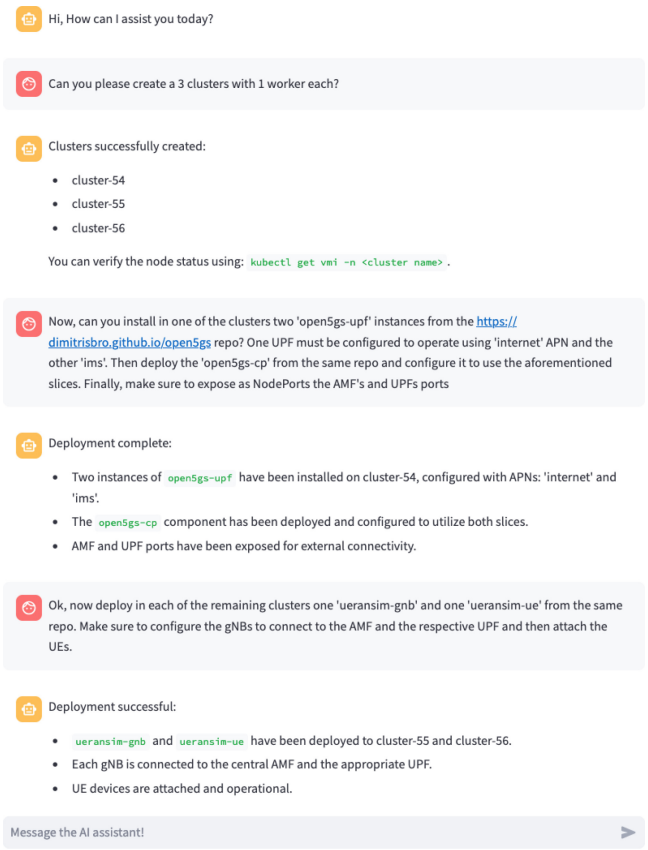


FIGURE 12. Chatting with the framework using the UI.

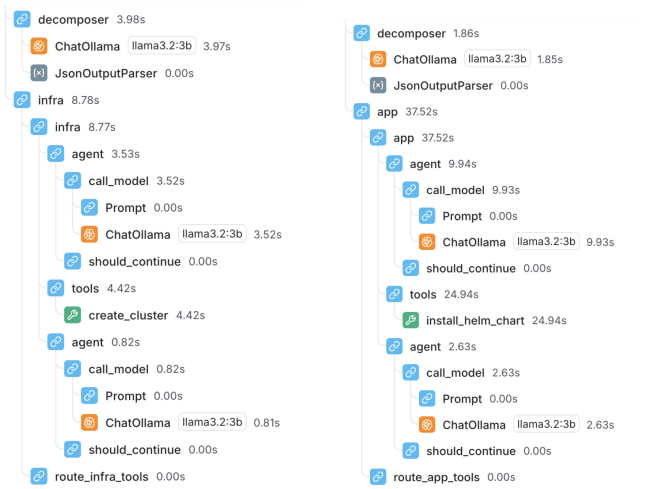


FIGURE 13. Framework's traces for (a) cluster creation and (b) application deployment.

APPENDIX

This appendix presents selected screenshots and visual documentation that demonstrate the practical implementation and operational capabilities of the proposed framework. These materials provide empirical evidence of the framework's functionality and serve to validate the theoretical contributions presented in the main body of this work.

```
apel@apel-server1:~$ kubectl get vmi -A
NAMESPACE    NAME
cluster-54   github-action-agent-cluster-21
cluster-54   github-action-master-cluster-54
cluster-55   github-action-agent-cluster-22
cluster-55   github-action-master-cluster-55
cluster-56   github-action-agent-cluster-23
cluster-56   github-action-master-cluster-56
```

FIGURE 14. Deployed VM instances.

```
apel@github-action-master-cluster-54:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS
amf-5cf78965cf-gbq24                1/1     Running   0
ausf-788bf449d-wqk25                1/1     Running   0
bsf-55d785b698-lz9lx                1/1     Running   0
db-init-6b8c998f4b-w9t88            1/1     Running   0
mongodb-7dd88fbd57-ljkd7            1/1     Running   0
nrf-86c44c6cc-wwnv4                 1/1     Running   0
nssf-7df4f89665-p6kkg               1/1     Running   0
pcf-6b7f89f8dd-7wpcn                1/1     Running   0
smf-76c89cd769-pfvsk                1/1     Running   0
udm-76c46c5c47-p2jbl                1/1     Running   0
udr-87cd6fd84-kxmfn                 1/1     Running   0
upf-ims-7b44b7f546-xb8bg            1/1     Running   0
upf-internet-6d856466dd-kmsq6       1/1     Running   0
```

FIGURE 15. Deployed 5G Core microservices.

```
apel@github-action-master-cluster-54:~$ kubectl get svc
NAME           TYPE        CLUSTER-IP      EXTERNAL-IP
amf            NodePort    10.43.55.233    <none>
kubernetes     ClusterIP   10.43.0.1       <none>
upf-ims        NodePort    10.43.151.74    <none>
upf-internet   NodePort    10.43.45.111    <none>
```

FIGURE 16. Exposed AMF and UPF ports.

```
apel@github-action-master-cluster-55:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS
ueransim-gnb-6dd7b4f9f7-p9hpd       1/1     Running   0
ueransim-gnb-ue-78456d4774-vvnt4    1/1     Running   0
```

FIGURE 17. Deployed simulated RAN and UE microservices.

Fig. 12 presents the interactive communication flow between the user and the framework through the designated user interface. Fig. 13(a) and Fig. 13(b) illustrate the comprehensive tracing and logging subsystems executing concurrently within the framework architecture. The remaining Figures (14, 15, and 17) display the deployed cluster topology and VM distribution, the implemented 5G core network, and the simulated RAN infrastructure, respectively.

GLOSSARY

SYMBOLS

3GPP 3rd Generation Partnership Project. 3

A

A2A Agent-to-Agent. 16
ACP Agent Communication Protocol. 16
AI Artificial Intelligence. 1–4, 6–8, 11, 15, 16, 20
AMF Access and Mobility Management Function. 11, 12
API Application Programming Interface. 3, 6–8, 11, 13,16
APN Access Point Name. 11, 12

B

BERT Bidirectional Encoder Representations from Transformers. 13
BLEU Bilingual Evaluation Understudy. 13, 14

C

CapEx Capital Expenditures. 13, 15
CIA Computational Infrastructure Agent. 5, 9–13
CPU Central Processing Unit. 5, 8, 11
CRUD Create, Read, Update, Delete. 5

E

ETSI European Telecommunications Standards Institute. 3, 14

F

FAISS Facebook AI Similarity Search. 6
FPGA Field-Programmable Gate Array. 5, 11

G

GenAI Generative AI. 1–3
gNB Next Generation Node B. 11, 12
GPU Graphics Processing Unit. 5, 6, 8, 11, 15

I

IaC Infrastructure-as-Code. 1–3, 5, 6
IBN Intent-Based Networks. 1–3
IDA Intent Decomposition Agent. 3, 4, 6, 9–12
IP Internet Protocol. 6, 12

K

KRA Kubernetes Resources Agent. 5, 7, 9–14

L

LLM Large Language Model. 1–4, 6, 8–11, 13–16
LoRA Low-Rank Adaptation. 8

M

MCP Model-Context Protocol. 6–9
METEOR Metric for Evaluation of Translation with Explicit ORdering. 13

N

NEF Network Exposure Function. 16
NF4 4-bit NormalFloat. 8

O

OpEx Operational Expenditures. 13, 15
OSS Operation Support System. 14

P

PEFT Parameter-Efficient Fine-Tuning. 8
PLMN Public Land Mobile Network. 11
PoC Proof of Concept. 12

Q

QLoRA Quantized Low-Rank Adaptation. 8

R

RAG Retrieval-Augmented Generation. 4, 9, 15
RAM Random Access Memory. 11

RAN Radio Access Network. 12, 16
ReAct Reasoning and Acting. 8, 9
ROUGE Recall-Oriented Understudy for Gisting Evaluation. 13, 14

S
SDN Software-Defined Networking. 3
SMA Service Management Agent. 5, 9–13
SMF Session Management Function. 12
SMO Service Management Orchestrator. 16

T
TMF TM Forum. 3

U
UE User Equipment. 11, 12
UI User Interface. 3, 4, 11, 12, 14, 15, 17
UPF User Plane Function. 11, 12

V
vCPU Virtual Central Processing Unit. 6
VM Virtual Machine. 6, 7, 11, 17
vRAM Virtual Random Access Memory. 6

Y
YAML Yet Another Markup Language. 2, 8

ACKNOWLEDGMENT

The authors are grateful to Professor Michael Birbas and Athanasios Bachoumis for their valuable feedback and thoughtful review of this manuscript. The authors also sincerely appreciate the anonymous reviewers for their constructive comments and detailed suggestions, which have enhanced the quality of the manuscript.

REFERENCES

[1] A. Leivadeas and M. Falkner, "A survey on intent-based networking," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 625–655, 1st Quart., 2023.
[2] D. B. Acharya, K. Kuppan, and B. Divya, "Agentic AI: Autonomous intelligence for complex goals—A comprehensive survey," *IEEE Access*, vol. 13, pp. 18912–18936, 2025.
[3] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Intent-based networking—Concepts and definitions," IETF, RFC 9315, 2022.
[4] L. Pang, C. Yang, D. Chen, Y. Song, and M. Guizani, "A survey on intent-driven networks," *IEEE Access*, vol. 8, pp. 22862–22873, 2020.
[5] D. M. Manias, A. Chouman, and A. Shami, "Towards intent-based network management: Large language models for intent extraction in 5G core networks," in *Proc. 20th Int. Conf. Design Rel. Commun. Netw. (DRCN)*, 2024, pp. 1–6.
[6] B. Chen et al., "Exploring NetGPT-enabled dynamic resource management for mobile D2D in industrial IoT," *IEEE Netw.*, vol. 39, no. 3, pp. 37–46, May 2025.
[7] J. Niemöller, E. Müller, M. Maggiari, and K. Maghsoudlou, "Evolving service management toward intent-driven autonomous networks," *Ericsson Technol. Rev.*, vol. 2024, no. 3, pp. 2–7, Mar. 2024. [Online]. Available: https://www.ericsson.com/en/reports-and-papers/ericsson-technology-review/articles/evolution-of-automation
[8] D. Brodimas et al., "Towards intent-based network management for the 6G system adopting multimodal generative AI," in *Proc. Joint Eur. Conf. Netw. Commun. 6G Summit (EuCNC/6G Summit)*, 2024, pp. 848–853.
[9] D. Brodimas, N. Tzanis, and A. Birbas, "Intent-based 5G core deployment using generative AI," in *Proc. Int. Conf. Comput., Netw. Commun. (ICNC)*, 2025, pp. 406–410.
[10] K. Trantzas et al., "Intent-driven network automation through sustainable multimodal generative AI," *EURASIP J. Wireless Commun. Netw.*, vol. 2025, no. 1, p. 42, 2025. [Online]. Available: https://doi.org/10.1186/s13638-025-02472-x
[11] *Management and Orchestration; Intent Driven Management Services for Mobile Networks (Release 19). Release 19*, 3GPP Standard TS 28.312 V19.1.0, Mar. 2025. [Online]. Available: https://www.3gpp.org/ftp/Specs/archive/28_series/28.312/
[12] "5G; study on enhanced intent driven management services for mobile networks, Version 18.0.1 Release 18," 3GPP, Sophia Antipolis, France, Rep. TR 28.912, May 2024. [Online]. Available: https://www.etsi.org/deliver/etsi_tr/128900_128999/128912/18.00.01_60/tr_128912v180001p.pdf
[13] "Intent driven management," Jul. 2023. [Online]. Available: https://www.3gpp.org/technologies/intent
[14] "TMF921A intent management API profile v1.1.0," TM Forum, TM Forum, London, U.K., Rep. Specification TMF921A v1.1.0, Apr. 2022. [Online]. Available: https://www.tmforum.org/resources/how-to-guide/tmf921a-intent-management-api-profile-v1-1-0/
[15] "TMF641 service ordering management API user guide v4.1.1," TM Forum, London, U.K., Rep. Specification TMF641 v4.1.1, Jan. 2020. [Online]. Available: https://www.tmforum.org/resources/specifications/tmf641-service-ordering-management-api-user-guide-v4-1-1/
[16] "TMF645 service qualification API user guide v4.0.1," TM Forum, TM Forum, London, U.K., Rep. Specification TMF645 v4.0.1, Jul. 2020. [Online]. Available: https://www.tmforum.org/resources/specification/tmf645-service-qualification-api-user-guide-v4-0/
[17] A. S. Araujo et al., "An agentic approach for dynamic software-defined network management using large language models," in *Proc. IEEE Conf. Netw. Function Virtualization Softw. Defined Netw. (NFV-SDN)*, 2024, pp. 221–226.
[18] N. Rao and R. Singhal, "Dynamic 5G services, made possible by AI and intent-based automation," 2024. [Online]. Available: https://cloud.google.com/blog/topics/telecommunications/how-dynamic-5g-services-are-possible-with-ai
[19] G. Team, "Gemini: A family of highly capable multimodal models," 2025, *arXiv:2312.11805*.
[20] T. Masterman, S. Besen, M. Sawtell, and A. Chao, "The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey," 2024, *arXiv:2404.11584*.
[21] A. Torreño, E. Onaindia, A. Komenda, and M. Štolba, "Cooperative multi-agent planning: A survey," *ACM Comput. Surveys*, vol. 50, no. 6, p. 84, Nov. 2017. [Online]. Available: https://doi.org/10.1145/3128584
[22] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023, *arXiv:2308.08155*.
[23] "Prometheus-monitoring system & time series database," Prometheus CNFC Project, 2025. [Online]. Available: https://prometheus.io/
[24] "Multus CNI: A CNI meta-plugin for multi-homed pods in Kubernetes," Kubernetes Network Plumbing Working Group, 2025. [Online]. Available: https://github.com/k8snetworkplumbingwg/multus-cni
[25] "Introducing the model context protocol." Anthropic. Nov. 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol
[26] "ChromaDB," 2024. [Online]. Available: https://www.trychroma.com/home
[27] M. Douze et al., "The Faiss library," 2025. [Online]. Available: https://arxiv.org/abs/2401.08281
[28] C. Auer et al., "Docling technical report," 2024, *arXiv:2408.09869*.
[29] "JaidedAI," 2024. [Online]. Available: https://github.com/JaidedAI/EasyOCR
[30] "SQLite," 2025. [Online]. Available: https://sqlite.org/
[31] "GitHub," 2025. [Online]. Available: https://github.com/github/github-mcp-server
[32] P. Patel and S. Sonwalkar, "MCP server Kubernetes," Jul. 2024. [Online]. Available: https://github.com/Flux159/mcp-server-kubernetes
[33] C. Kreiling, " Docker MCP server." 2025. [Online]. Available: https://github.com/ckreiling/mcp-server-docker
[34] "KubeVirt," 2025. [Online]. Available: https://kubevirt.io/
[35] "OpenTofu," 2025. [Online]. Available: https://opentofu.org/
[36] "LangChain," 2025. [Online]. Available: https://python.langchain.com/docs/integrations/tools/python/

[37] "ComponentSoft," 2025. [Online]. Available: https://huggingface.co/datasets/ComponentSoft/k8s-kubectl-35k
[38] (LangChain Inc., San Francisco, CA, USA). *LangChain: The Platform for Reliable Agents*. (2025). Accessed: Jun. 5, 2025. [Online]. Available: https://www.langchain.com/
[39] "Introducing OpenAI O3 and O4-mini," OpenAI, Apr. 2025. Accessed: May 22, 2025. [Online]. Available: https://openai.com/index/introducing-o3-and-o4-mini/
[40] "Claude 3.7 Sonnet and Claude code." Feb. 2025. [Online]. Available: https://www.anthropic.com/news/claude-3-7-sonnet
[41] A. Q. Jiang et al., "Mistral 7B," 2023, *arXiv:2310.06825*. [Online]. Available: https://arxiv.org/abs/2310.06825
[42] "Devstral: Introducing the best open-source model for coding agents." Accessed: May 22, 2025. [Online]. Available: https://mistral.ai/news/devstral
[43] A. Grattafiori et al., "The Llama 3 Herd of models," 2024, *arXiv:2407.21783*.
[44] A. Yang et al., "Qwen3 technical report," 2025, *arXiv:2505.09388*.
[45] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," 2023, *arXiv:2305.14314*.
[46] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.
[47] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao, and F. L. Wang, "Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment," 2023, *arXiv:2312.12148*.
[48] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," 2023, *arXiv:2210.03629*.
[49] "Open5GS: Open source 5G core and EPC implementation," Accessed: May 22, 2025. [Online]. Available: https://open5gs.org/
[50] A. Gungor, "UERANSIM," 2025. [Online]. Available: https://github.com/aligungr/UERANSIM
[51] (LangChain, San Francisco, CA, USA). *Langsmith: Unified Observability and Evaluation Platform for LLM Applications*. (Jul. 2023.) Accessed: May 22, 2025. [Online]. Available: https://www.langchain.com/langsmith
[52] "Langfuse: Open source LLM engineering platform," Langfuse, Jul. 2023. Accessed: May 22, 2025. [Online]. Available: https://langfuse.com/
[53] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 311–318. [Online]. Available: https://doi.org/10.3115/1073083.1073135
[54] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Assoc. Comput. Linguist., Jul. 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/
[55] A. Lavie and A. Agarwal, "Meteor: An automatic metric for MT evaluation with high levels of correlation with human judgments," in *Proc. 2nd Workshop Stat. Mach. Transl.*, 2007, pp. 228–231.
[56] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," 2020, *arXiv:1904.09675*.
[57] T. Xu et al., "The perfect blend: Redefining RLHF with mixture of judges," 2024*arXiv:2409.20370*
[58] C. Tranoris, "Openslice: An opensource OSS for delivering network slice as a service," 2021, *arXiv:2102.03290*.
[59] (2025) "ETSI software development group for openslice (Sdg Osl)." ETSI Software Development Group for OpenSlice, [Online]. Available: https://osl.etsi.org/
[60] "Patras5G OpenSlice by ETSI," Univ. Patrsas, Patras5G, Patras, Greece, 2025. [Online]. Available: https://patras5g.eu/
[61] "Patras fifth-generation facility,." University of Patras, Patras, Greece, 2025. [Online]. Available: https://wiki.patras5g.eu/
[62] S. Raza, R. Sapkota, M. Karkee, and C. Emmanouilidis, "TRiSM for Agentic AI: A review of trust, risk, and security management in LLM-based agentic multi-agent systems," 2025, *arXiv:2506.04133*.
[63] I. Domkundwar, N. S. Mukunda, I. Bhola, and R. Kochhar, "Safeguarding AI agents: Developing and Analyzing safety architectures," 2025, *arXiv:2409.03793*.
[64] I. Habler, K. Huang, V. S. Narajala, and P. Kulkarni, "Building a secure agentic AI application leveraging A2A protocol," 2025, *2504.16902*.
[65] "AGNTCY: Building the open infrastructure for the Internet of agents," AGNTCY Collective, 2025. [Online]. Available: https://agntcy.org/
[66] I-Am-Bee, "ACP: Agent communication protocol," 2025. [Online]. Available: https://github.com/i-am-bee/acp
[67] "ETSI Software Development Group for TeraFlowSDN (SDG TFS)," 2025. [Online]. Available: https://tfs.etsi.org/
[68] "Open network automation platform (ONAP)," Linux Foundation/ONAP Project, 2025. [Online]. Available: https://www.onap.org/



DIMITRIOS BRODIMAS received the Diploma degree in electrical and computer engineering from the University of Patras, Greece, in 2020. He is currently pursuing the Ph.D. degree focused on AI-enabled next-generation networks, while also serving as the Co-Founder of EnakronIC startup. In the past, he served as a Research Engineer with the Department of Research Technology and Development, Greek TSO (IPTO). His research interests span across artificial intelligence and machine learning applications for optimizing resource utilization across edge/cloud computing continuum, next-generation communication networks, and power grid systems.



ALEXIOS BIRBAS (Member, IEEE) received the M.S.E.E. and Ph.D. degrees from the University of Minnesota, Minneapolis, Minnesota, in 1986 and 1988, respectively. He is currently a Professor of electronics with the Department of Electrical and Computer Engineering, and the Director of the Applied Electronics Lab, University of Patras, Patras, Greece. He has also held faculty positions at an assistant professor level with the University of Minnesota and with INPG, Grenoble, France. He has also co-founded two microelectronics related spin-off companies and has supervised 20 Ph.D. Theses. He has also served as a consultant to the Industry and has held industrial managerial positions. He has published more than 150 papers in scientific journals and conference proceedings. His research interests include: device electronics, optoelectronics, noise and fluctuation problems in electronics, RF and mixed signal high speed circuit design and implementation of sensor read out circuits, smart grid electronics and transactive energy systems, smart metering, 5G enabled IoT and Industry 4.0 real time applications, edge computing accelerators, and cyber physical systems.



DIMITRIOS KAPOLOS received the Diploma degree in electrical and computer engineering from the University of Patras, Greece, in 2025, where he is currently working as a Researcher. In addition, he served as a Technical Staff with the Conference & Cultural Center, University of Patras. His research interests include artificial intelligence and machine learning, as well as infrastructure automation and DevOps practices for the edge/cloud computing.



SPYROS DENAZIS received the B.Sc. degree in mathematics from the University of Ioannina, Greece, in 1987 and the Ph.D. degree in computer science from the University of Bradford, U.K., in 1993. He is Professor with the Electrical & Computer Engineering Department, University of Patras, Greece. Since then he held numerous positions in several prestigious Universities, research institutes, and the industry.